

# Содержание

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| 1. Библиотека <code>numpy</code> . Основные функции .....                     | 2  |
| 2. Библиотека <code>matplotlib</code> . Основные функции, типы диаграмм ..... | 2  |
| 3. Метод половинного деления .....                                            | 3  |
| 4. Метод итераций .....                                                       | 5  |
| 5. Метод Ньютона .....                                                        | 6  |
| 6. Метод Гаусса .....                                                         | 6  |
| 7. Метод Якоби и метод Зейделя .....                                          | 7  |
| Метод Якоби .....                                                             | 8  |
| Метод Зейделя .....                                                           | 8  |
| 8. Метод прямоугольников и трапеций .....                                     | 9  |
| Метод прямоугольников .....                                                   | 9  |
| Метод трапеций .....                                                          | 10 |
| 9. Метод Симпсона .....                                                       | 10 |
| 10. Полиномиальная интерполяция функций .....                                 | 11 |
| 11. Линейная регрессия .....                                                  | 12 |
| 12. Градиент функции .....                                                    | 13 |
| 13. Оценка погрешности численных методов .....                                | 14 |

# 1. Библиотека `numpy`. Основные функции

`numpy` — это библиотека Python для работы с числами, массивами и матрицами. Обычный список Python просто хранит значения, а `numpy` позволяет быстро выполнять вычисления сразу со всем массивом.

Например, если умножить массив на 2, умножится каждый его элемент:

```
import numpy as np

a = np.array([1, 2, 3])
print(a * 2)  # [2 4 6]
```

Главный объект в `numpy` — массив `ndarray`. Его можно представить как таблицу чисел: в одну строку, в несколько строк или даже в несколько измерений.

Основные функции:

- `np.array()` — создать массив;
- `np.zeros()` — создать массив из нулей;
- `np.ones()` — создать массив из единиц;
- `np.arange()` — создать последовательность чисел с шагом;
- `np.linspace()` — разделить отрезок на равные части;
- `np.sum()`, `np.mean()`, `np.min()`, `np.max()` — найти сумму, среднее, минимум и максимум;
- `np.sqrt()`, `np.sin()`, `np.cos()`, `np.exp()` — выполнять математические операции;
- `np.linalg.solve()` — решать системы линейных уравнений.

Пример решения системы:

```
A = np.array([[2, 1], [1, 3]])
b = np.array([5, 6])

x = np.linalg.solve(A, b)
```

В численных методах `numpy` нужен почти постоянно, потому что многие задачи сводятся к вычислениям с массивами, таблицами и матрицами.

## 2. Библиотека `matplotlib`. Основные функции, типы диаграмм

`matplotlib` — это библиотека Python для построения графиков. Она нужна, когда хочется не только получить числа, но и увидеть результат глазами: график функции, точки, диаграмму или распределение данных.

Чаще всего используют модуль `pyplot`:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(-5, 5, 100)
y = x**2
```

```
plt.plot(x, y)
plt.grid()
plt.show()
```

Основные виды графиков:

- `plt.plot()` — обычный график линии, например график функции;
- `plt.scatter()` — точки на плоскости;
- `plt.bar()` — столбчатая диаграмма;
- `plt.hist()` — гистограмма, то есть распределение значений;
- `plt.pie()` — круговая диаграмма;
- `plt.imshow()` — показ изображения или матрицы.

Для оформления графика используют:

- `plt.title()` — название графика;
- `plt.xlabel()` и `plt.ylabel()` — подписи осей;
- `plt.legend()` — легенда;
- `plt.grid()` — сетка;
- `plt.show()` — показать график;
- `plt.savefig()` — сохранить график в файл.

В численных методах `matplotlib` удобно использовать, чтобы показать, где функция пересекает ось, как меняется ошибка, как выглядят данные или насколько хорошо модель описывает точки.

### 3. Метод половинного деления

Метод половинного деления, или метод бисекции, нужен для приближённого решения уравнения:

$$f(x) = 0$$

Проще говоря, мы ищем такое значение  $x$ , при котором функция становится равной нулю. На графике это точка, где кривая пересекает ось  $Ox$ .

Метод работает так: берём отрезок  $[a; b]$ , на концах которого функция имеет разные знаки. То есть с одной стороны значение функции отрицательное, а с другой положительное:

$$f(a) \cdot f(b) < 0$$

Если функция непрерывная, значит где-то между  $a$  и  $b$  она точно проходит через ноль.

Алгоритм:

1. Берём середину отрезка:

$$c = \frac{a + b}{2}$$

2. Считаем значение  $f(c)$ .

3. Смотрим, в какой половине отрезка осталась смена знака.

4. Оставляем только эту половину.

5. Снова делим её пополам.

6. Повторяем, пока не получим нужную точность.

Пример:

$$f(x) = x^2 - 2$$

На отрезке  $[1; 2]$ :

$$f(1) = -1, \quad f(2) = 2$$

Знаки разные, значит корень внутри отрезка есть. После нескольких делений получится:

$$x \approx 1.414$$

Это приближённое значение  $\sqrt{2}$ .

Погрешность метода после  $n$  шагов можно оценить так:

$$|x - x_n| \leq \frac{b - a}{2^n}$$

Главный плюс метода — он надёжный. Если есть смена знака и функция непрерывная, корень будет найден. Минус — работает не очень быстро.

## 4. Метод итераций

Метод простых итераций тоже используется для решения уравнения:

$$f(x) = 0$$

Но сначала уравнение нужно переписать в виде:

$$x = \varphi(x)$$

Смысл такой: мы берём первое приближение  $x_0$ , подставляем его в правую часть и получаем новое значение:

$$x_{n+1} = \varphi(x_n)$$

То есть каждое новое значение считается по предыдущему. Если значения постепенно приближаются к одному числу, это число и будет решением.

Пример:

$$x^2 - 2 = 0$$

Можно использовать формулу:

$$x_{n+1} = \frac{x_n + \frac{2}{x_n}}{2}$$

Пусть  $x_0 = 1$ . Тогда:

$$x_1 = 1.5$$

$$x_2 \approx 1.4167$$

$$x_3 \approx 1.4142$$

Получаем:

$$\sqrt{2} \approx 1.4142$$

Графически метод можно понять так: решение — это точка пересечения графиков  $y = x$  и  $y = \varphi(x)$ .

Чтобы метод хорошо сходился, обычно нужно условие:

$$|\varphi'(x)| < 1$$

Останавливают вычисления, когда соседние значения почти не отличаются:

$$|x_{n+1} - x_n| < \varepsilon$$

Главный минус метода: если плохо выбрать функцию  $\varphi(x)$ , значения могут не приближаться к ответу, а наоборот расходиться.

## 5. Метод Ньютона

Метод Ньютона, или метод касательных, решает уравнение:

$$f(x) = 0$$

Формула метода:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'}(x_n)$$

Идея такая: мы берём точку на графике функции, проводим в ней касательную и смотрим, где эта касательная пересекает ось Ох. Эта точка пересечения становится новым приближением корня.

Пример:

$$f(x) = x^2 - 2$$

Производная:

$$f'(x) = 2x$$

Формула Ньютона:

$$x_{n+1} = x_n - \frac{x_n^2 - 2}{2x_n}$$

Если взять  $x_0 = 1.5$ , то:

$$x_1 \approx 1.4167$$

$$x_2 \approx 1.4142$$

То есть очень быстро получаем приближение к  $\sqrt{2}$ .

Плюс метода Ньютона — он обычно сходится быстро. Минусы: нужно знать производную, нужно выбрать нормальное начальное приближение, а если производная близка к нулю, метод может дать плохой результат.

## 6. Метод Гаусса

Метод Гаусса используется для решения систем линейных уравнений. В общем виде систему можно записать так:

$$Ax = b$$

Смысл метода: мы постепенно преобразуем систему так, чтобы она стала проще. Сначала приводим её к треугольному виду, а потом находим неизвестные снизу вверх.

Пример:

$$2x + y = 5$$

$$x + 3y = 6$$

Сначала нужно избавиться от  $x$  во втором уравнении. После преобразования получаем:

$$2x + y = 5$$

$$2.5y = 3.5$$

Теперь второе уравнение легко решить:

$$y = 1.4$$

Подставляем это значение в первое уравнение:

$$2x + 1.4 = 5$$

$$x = 1.8$$

Ответ:

$$x = 1.8, \quad y = 1.4$$

Метод Гаусса состоит из двух частей:

- прямой ход — зануляем коэффициенты под главной диагональю;
- обратный ход — находим неизвестные, начиная с последнего уравнения.

Чтобы уменьшить ошибки вычислений, часто используют выбор главного элемента. Это значит, что на каждом шаге выбирают строку с самым удобным, обычно самым большим по модулю, коэффициентом.

## 7. Метод Якоби и метод Зейделя

Методы Якоби и Зейделя — это итерационные методы решения систем линейных уравнений:

$$Ax = b$$

Они работают не как метод Гаусса. Здесь мы не решаем систему сразу, а постепенно улучшаем приближённый ответ.

Пример системы:

$$10x + y = 11$$

$$2x + 10y = 12$$

Выразим переменные:

$$x = \frac{11 - y}{10}$$

$$y = \frac{12 - 2x}{10}$$

## Метод Якоби

В методе Якоби все новые значения считаются только через старые значения.

Пусть сначала:

$$x_0 = 0, \quad y_0 = 0$$

Тогда первая итерация:

$$x_1 = \frac{11 - 0}{10} = 1.1$$

$$y_1 = \frac{12 - 2 * 0}{10} = 1.2$$

То есть  $x_1$  и  $y_1$  считаются независимо друг от друга, оба через старые значения.

## Метод Зейделя

В методе Зейделя новое значение используется сразу.

Сначала считаем:

$$x_1 = 1.1$$

А потом при вычислении  $y_1$  уже используем это новое  $x_1$ :

$$y_1 = \frac{12 - 2 * 1.1}{10} = 0.98$$

Из-за этого метод Зейделя часто сходится быстрее метода Якоби.



Хорошее условие для сходимости — диагональное преобладание. Простыми словами: в каждом уравнении коэффициент при основной переменной должен быть больше остальных коэффициентов в этой строке:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

Останавливают метод, когда новое приближение почти не отличается от старого:

$$\max_i |x_i^{(k+1)} - x_i^k| < \varepsilon$$

## 8. Метод прямоугольников и трапеций

Численное интегрирование нужно, чтобы приближённо найти определённый интеграл:

$$\int_a^b f(x) dx$$

Графически интеграл — это площадь под графиком функции от  $a$  до  $b$ .

### Метод прямоугольников

Идея простая: сложную площадь под кривой заменяем набором прямоугольников. Чем больше прямоугольников, тем точнее результат.

Отрезок  $[a; b]$  делится на  $n$  частей. Шаг:

$$h = \frac{b - a}{n}$$

Формула левых прямоугольников:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{n-1} f(x_i)$$

Формула правых прямоугольников:

$$\int_a^b f(x) dx \approx h \sum_{i=1}^n f(x_i)$$

Формула средних прямоугольников:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{n-1} f\left(\frac{x_i + x_{i+1}}{2}\right)$$

Метод средних прямоугольников обычно точнее левых и правых, потому что берёт значение функции в середине каждого маленького отрезка.

## Метод трапеций

Метод трапеций похож, но вместо прямоугольников используются трапеции. На каждом маленьком отрезке график функции заменяется прямой линией.

Формула:

$$\int_a^b f(x) dx \approx h \left( \frac{f(x_0) + f(x_n)}{2} + \sum_{i=1}^{n-1} f(x_i) \right)$$

Графически это значит, что мы заменяем площадь под кривой суммой площадей трапеций. Обычно метод трапеций точнее метода прямоугольников, потому что он учитывает наклон графика.

## 9. Метод Симпсона

Метод Симпсона — это метод численного интегрирования, который обычно точнее прямоугольников и трапеций.

Идея такая: на маленьких участках график функции заменяется не прямой линией, а параболой. Парабола лучше повторяет форму кривой, поэтому результат часто получается точнее.

Нужно вычислить:

$$\int_a^b f(x) dx$$

Отрезок делится на  $n$  равных частей, причём  $n$  обязательно должно быть чётным. Шаг:

$$h = \frac{b - a}{n}$$

Формула Симпсона:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left( f(x_0) + f(x_n) + 4 \sum_{i=1,3,\dots,n-1} f(x_i) + 2 \sum_{i=2,4,\dots,n-2} f(x_i) \right)$$

Коэффициенты в формуле идут так:

$$1, 4, 2, 4, 2, \dots, 4, 1$$

Пример:

$$\int_0^2 x^2 dx$$

Если взять  $n = 2$ , то:

$$h = 1$$

$$I \approx \frac{1}{3}(f(0) + 4f(1) + f(2))$$

$$I \approx \frac{1}{3}(0 + 4 + 4) = \frac{8}{3}$$

В этом примере метод даёт точный ответ, потому что функция  $x^2$  сама является параболой.

## 10. Полиномиальная интерполяция функций

Интерполяция нужна, когда у нас есть несколько известных точек, а нужно примерно понять, какое значение функция имеет между ними.

Пусть известны точки:

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$$

Нужно построить многочлен  $P(x)$ , который проходит через эти точки:

$$P(x_i) = y_i$$

Самый простой вариант — линейная интерполяция. Она просто соединяет две точки прямой.

Формула:

$$y = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0)$$

Пример:

Есть точки  $(1, 2)$  и  $(2, 4)$ . Нужно найти значение при  $x = 1.5$ :

$$y = 2 + \frac{4 - 2}{2 - 1}(1.5 - 1) = 3$$

Значит, между этими точками при  $x = 1.5$  получаем  $y = 3$ .

Для большего количества точек можно использовать многочлен Лагранжа:

$$P(x) = \sum_{i=0}^n y_i L_{i(x)}$$

Интерполяция применяется при работе с табличными данными, построении графиков, приближённом восстановлении функции и вычислении промежуточных значений.

Важно: если взять слишком много точек и построить один большой многочлен, он может начать сильно колебаться. Поэтому на практике часто используют кусочную интерполяцию.

## 11. Линейная регрессия

Линейная регрессия — это способ провести прямую через набор точек так, чтобы она как можно лучше описывала зависимость.

Простейшая модель:

$$y = kx + b$$

Здесь  $k$  показывает наклон прямой, а  $b$  показывает, где прямая пересекает ось  $y$ .

Графический смысл простой: есть точки на графике, и мы хотим провести через них одну прямую. Она не обязана проходить через каждую точку точно, но должна проходить так, чтобы ошибки были как можно меньше.

Пример данных:

| $x$ | $y$ |
|-----|-----|
| 1   | 2   |
| 2   | 4   |
| 3   | 6   |

Здесь подходит зависимость:

$$y = 2x$$

Если  $x = 4$ , то модель предскажет:

$$y = 8$$

Обычно регрессия подбирает прямую так, чтобы средняя ошибка была минимальной. Часто используют среднеквадратичную ошибку:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Если признаков несколько, модель записывают так:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Линейная регрессия используется для прогнозов: например, цены, спроса, оценки, времени выполнения задачи. Но она хорошо работает только тогда, когда зависимость примерно похожа на прямую.

## 12. Градиент функции

Градиент — это вектор, который показывает, куда функция растёт быстрее всего.

Если функция зависит от двух переменных, например  $f(x, y)$ , то градиент состоит из двух частных производных:

$$\nabla f = \left( \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

Пример:

$$f(x, y) = x^2 + y^2$$

Производные:

$$\frac{\partial f}{\partial x} = 2x$$

$$\frac{\partial f}{\partial y} = 2y$$

Значит:

$$\nabla f = (2x, 2y)$$

В точке  $(3, 4)$ :

$$\nabla f = (6, 8)$$

Это значит, что в этой точке функция быстрее всего растёт в направлении вектора  $(6, 8)$ .

Для поиска минимума идут в обратную сторону от градиента. Это называется градиентный спуск:

$$x_{\text{new}} = x_{\text{old}} - \alpha \nabla f(x_{\text{old}})$$

Здесь  $\alpha$  — это размер шага. Если шаг слишком большой, можно перескочить минимум. Если слишком маленький, метод будет работать медленно.

Пример для одной переменной:

$$f(x) = x^2$$

$$f'(x) = 2x$$

Пусть  $x_0 = 5$ , а  $\alpha = 0.1$ . Тогда:

$$x_1 = 5 - 0.1 * 2 * 5 = 4$$

Дальше значение будет постепенно приближаться к 0, а у функции  $x^2$  минимум как раз находится в точке  $x = 0$ .

### 13. Оценка погрешности численных методов

Погрешность показывает, насколько наш приближённый ответ отличается от точного.

Если точное значение —  $x$ , а приближённое —  $x^*$ , то абсолютная погрешность:

$$\Delta = |x - x^*|$$

Она показывает сам размер ошибки.

Относительная погрешность:

$$\delta = \frac{|x - x^*|}{|x|}$$

Она показывает ошибку относительно самого значения. В процентах:

$$\delta_{\%} = \frac{|x - x^*|}{|x|} * 100\%$$

Пример:

Пусть точное значение  $x = 10$ , а приближённое  $x^* = 9.8$ .

Абсолютная погрешность:

$$\Delta = |10 - 9.8| = 0.2$$

Относительная погрешность:

$$\delta = \frac{0.2}{10} = 0.02 = 2\%$$

Основные виды погрешностей:

- погрешность исходных данных — если начальные данные уже неточные;

- погрешность округления — появляется из-за ограниченной точности вычислений;
- погрешность метода — возникает потому, что сам метод приближённый;
- погрешность итераций — появляется, если остановить вычисления раньше точного результата.

Для итерационных методов часто используют критерий остановки:

$$|x_{n+1} - x_n| < \varepsilon$$

То есть если новое значение почти не отличается от старого, вычисления можно остановить.

Для метода половинного деления оценка такая:

$$|x - x_n| \leq \frac{b - a}{2^n}$$

Ещё важны два свойства:

- сходимость — метод постепенно приближается к правильному ответу;
- устойчивость — небольшая ошибка в данных не ломает весь результат.